

Video Transcript

Video 4: Flask Server Registration and Login (17:09)

We've already seen how to set up a simple web server using Flask, and that web server had two routes; It had the default route, the slash here, so localhost 5000 slash would take us here, or the books route. So, localhost 5000 slash books would take us here and it would fire the functions that were on those routes. So, in the Flask code there are functions that are under the decorator and it fires those. Now, what this means is that from this browser we can control which function is fired on this web server. This is actually distributed computing.

This was a major challenge for many years in the 1990s of how we could do cross machine computations, how we could fire functions on another machine and have them return a result to us. Now, Sir Tim Berners Lee solved this in a very neat way by using this HTTP request and response. So, we can make a request on some route which will fire a function and return to us some result, an HTTP response. That's pretty amazing. And we'll come back to that because it's at the basis of most of the computing, the web computing that's going on today. But for now, we're going to make our web server a little bit more realistic.

One of the things that we'd like to do is when a user hits us, we'd like to have control over what routes they can visit or over what functions they can fire. So, we're going to force them to log in. Now, we don't want them to log in every time they visit us. So, what we're going to do is when they issue an HTTP request, we're going to check if they've registered or not. So, if they haven't logged in, we're going to send them to this register page. We're going to return to them register.html and what that is a form that will ask them their username and their password. So, when they submit that form, the action on the form is to go to slash login. So, once they click 'Submit', that form will be passed to slash login and slash login then will pick out their username and password and check whether that matches the usernames and passwords that they allow to log in. So, we're going to control the user now, and if they've only, if they've got the right username and password, will we let them see our books, for example. Now, we're going to use session cookies, sometimes called session IDs, to track the users. We're not going to ask them to log in every time, we're going to put a cookie on their machine.

So, once they've logged in, we're going to send back a response that has a cookie in it that will be then placed on that browser. And every time that browser then makes a request, it'll pass that cookie in the request back to us. We're going to check that cookie, and if it's valid, then we're going to let them do what they're allowed to do on our machine. So, this is the security that we need when we're talking to remote browsers. And we don't necessarily know who they are, so we have to validate them in some way. Okay, let's go and take a look at the code. So, first let's create a login or register HTML page. So, we need to have a form where the Username and password can be inputted, and then when we submit we want the action to be to go to /login to that root. Again, we're going to extend the underlying index.html form and we're going to insert this in the block content. So, here, this is going to replace on the index file. It's going to replace this block content down here.

Now, we've got three roots, and it's time we put in a navigation bar. So, here I'm putting in a nav element, and I'm putting in an href, and going if we click on this we'll go to slash login and if we click on this we'll go to slash books. So, now our page will look a little different. Let's take a quick look. What we'll have is we'll have register up here and list books here. So, this will be our navigation bar and we'll make it much prettier later on. But for now, we just want that functionality. And so, when we click on 'Register', we'll come to this form and I've set up a user called testuser so that, when we type that in and submit it, we'll be able to recognize that username and password.

Let's go back to the code. So, now we've got this navigation bar that will be, remember this is the base template indexed of HTML. So, all the others inherit this. So, this little navigation bar will appear on books and on register. So now, let's go to the app and we need to start putting in modifications here. So, one of the things that we're going to need to do is we're going to create a session ID cookie. And to do that we're going to need to store a secret key. And the way we do that is in app we define secret key, and this is arbitrary. If this was a production system, we'd put a much longer key in there and one that couldn't be guessed. But for now, I'm just putting secret key. Now, let's go and take a look. We need to input a user. So, here I'm going to input and I'm going to make a list for a number of users. I'm only going to put one in at the moment and I'm going to put that in as a dictionary in this list. So, the outer brackets of the list. And here I've got the dictionary and I'm going to have username and that's going to be test user and password test user. So, I'll only have one user.

Now I need to be able to check the user is going to register, they're going to input their username and password and I need to check if it's amongst the users that I recognize. So, I need some code to check that. So, here I've got a `checkUser`. If username and password is given to me, I'm going to loop over all the users in `user`. So, I'm going to check if username in `user` and if password is in `user` then I'm going to return true otherwise I'll return false. So, this is a simple check that I know the user's username and password.

We already had the default route. Now, I want to put in a login route. So, here I'm going to have `login slash login` and remember this is where I come, when I submit that form and that form will be posted. So, I need to support the POST method and I'm going to support the GET method as well. But that's just so that I can redirect somebody. They should be posting me their username and password. I can check the method. Flask provides a request function and part of that is the method, and I can check that it's a POST. Now, let's take a look at what we need to do here. Assuming that it's a POST, I'm going to pick up from the form. So, from the `request.form`, I'm going to pick up the username and the password. So, this is the way that we get the contents of a form that's passed to us. So, now I want to check if that username and password exist. If they do, I'm going to come inside this if loop. And now, what I'm going to do is I'm going to set something called session. I'm going to set and you can use any key here. I want to use username because I'm going to store the username in there. We could store an object in there, for example, I could have the username and I could have their role. Later on we may want that because not everyone should have access to everything, but for now I'm just going to set up session with username. Now, the user is logged in, I'm going to send them back to the index of HTML template and I'm going to pass in their username. OK, If they don't pass the check for a username and password, we're going to send them back to register. And now, if it's not a POST method, it must be a GET method because these are the only two we support. And if they do anything else, if they send us an update or a delete, then we'll just reject it.

At this level, this decorator will reject any other HTTP request, but here if it's a GET, we send them back to register because they haven't filled in the form. OK, so that's login and I'm providing a logout where we'll just destroy that session cookie. And as usual, we've got the books, and here we're just printing out the books, so let's save that. And one more thing to notice is that if they hit the default route, I'm passing them to the register template. OK, so let's run this code. It's already running. So, let's go now to here. And we hit local host 5000. Now you see our navigation

bar. It's not a very pretty one, but it's a functional one and we can now put in a user, test user, and test user and we can submit it.

Let me put in a break point before we submit it so we can see what's happening. So, I'm going to go in here and in login I'm going to put a break point. Now let's step. Let's submit, and we immediately come to login. Now let's step through. So, the request method is a POST. Now we're going to try and get the username and password, username test user, password test user. Now it should check out. Let's go through. We'll see we're looping over users in users and we're returning true because there's only one user at the moment and we pick them out and it's passes our test. So, now we step in here. Now we set the session cookie, We're setting username in the session cookie as our username, the one that we got from the form. Now we're going to go back to render the index template. And so we go back to our form and you'll see welcome test user and you'll see our navbar here where now we can list books. So, this part is coming from index.html, this is coming from books.html. And we've still got the register one. And if I click on that, it'll take me back around again here. So, we may want to, since we're already have registered, we may want to check that if they come to register again, that we don't want them to actually register. So, we may not want to put a if they already have a session cookie, maybe we don't have them re register again. Let's take a look at the session cookies. So, I want to go back to list books here.

Here we are list books. Now let me go and bring up the ID in Chrome. So, I'm going to do control option I on the Mac, control shift I on the PC and let me now I'm going to let me scrunch down this one so we can see what's happening here. So, now let me replay this. We'll see there's a message. It's hit books. Let's take a look. Now, I'm going to expand this so we can see the messages. So, in books, we can take a look at the headers, but I'm interested in the cookies.

And here you see our session cookie. This is the cookie we set and it looks, it's like quite long and it looks like it's encoded. And we can take a look at it later to see its structure. We'll take a look at cookies in some detail to describe them, but we'll see here that with the books, it was a GET. And our response, well, it had the cookie. It had the cookie already in there. We saw that, and then the response message was the books. OK, so every time now that the browser hits the web server, the browser passes the cookie it's stored in the browser.

So, the browser is storing now our cookie and passing it back to us each time it makes a request to the web server. And we'll talk later about exactly why we're doing this. It's because we need

our web server to scale, and by doing this we allow it to not have to keep track of the conversation with the user. So, it means that our web server can handle potentially hundreds of thousands of users. OK, so that's how we place a cookie. We'll see that there are more secure methods than this one, but this one is why a lot of companies are placing cookies on your browser, because they can then track you. OK, bye for now.

